



Universidade Federal do Ceará

Campus de Quixadá
Curso de Ciência da Computação

Trabalho I – Análise Léxica

Compiladores

Prof. Lucas Ismaily

Equipe:

Quixadá, 27 de junho de 2026

Sumário

1	Introdução	3
2	A linguagem LangC	3
2.1	Alfabeto	3
2.2	Tipos de dados	3
2.3	Estrutura do programa	4
2.3.1	Declaração de variável	4
2.3.2	Saída com show	4
3	Questão 1 – Tokens e expressões regulares	4
3.1	Definição dos tokens	4
3.2	Prioridade dos tokens	5
3.3	Implementação	5
4	Questão 2 – Construção do NFA	6
4.1	Construção de Thompson	6
4.1.1	NFA básico para um caractere	6
4.1.2	Concatenação	6
4.1.3	União	7
4.1.4	Fecho de Kleene (*) e Uma ou mais repetições (+)	7
4.2	Unificação dos NFAs	7
4.3	Resultado	7
5	Questão 3 – Conversão NFA para DFA	8
5.1	Algoritmo de subconjuntos	8
5.2	Epsilon-Fechamento	9
5.3	Resolução de conflitos por prioridade	9
5.4	Resultado	9
6	Questão 4 – Analisador léxico	10
6.1	Funcionamento	10
6.2	Validação adicional	10
6.3	Interface com o usuário	10
7	Testes e resultados	11
7.1	Casos de Teste	11
7.2	Análise dos resultados	11
7.2.1	Erros léxicos corretos	11
7.2.2	Prioridade de palavras reservadas	11

7.2.3	Maior casamento possível	12
7.2.4	Erros semânticos não tratados no léxico	12
7.3	Saída completa dos testes	12
8	Estrutura do projeto	13
9	Como executar	14
9.1	Requisitos	14
9.2	Arquivos necessários	14
9.3	Execução	14
9.4	Opção 1 – Carregar arquivo	15
9.5	Opção 2 – Modo manual	15
9.6	Opção 3 – Sair	16
10	Conclusão	16

1 Introdução

Este relatório descreve o desenvolvimento do Trabalho I da disciplina de Compiladores, referente à **Opção 1 – Análise Léxica** da linguagem **LangC**.

A LangC é uma linguagem minimalista, declarativa e estruturada, inspirada em C. Ela possui sintaxe simples, poucas palavras reservadas e regras rígidas. O objetivo deste trabalho é implementar um analisador léxico completo para essa linguagem, passando pelas seguintes etapas:

1. Definição dos tokens e suas expressões regulares
2. Construção do Autômato Finito Não-Determinístico (NFA) via Construção de Thompson
3. Conversão do NFA para Autômato Finito Determinístico (DFA) via Algoritmo de Subconjuntos
4. Implementação do analisador léxico utilizando o DFA

2 A linguagem LangC

2.1 Alfabeto

A linguagem aceita os seguintes símbolos:

- Letras: a-z, A-Z
- Dígitos: 0-9
- Espaço, tabulação e quebra de linha
- Símbolos reservados: + - * / = > < () ; " _ ! @ # \$ % & ? |

2.2 Tipos de dados

Tabela 1: Tipos de dados da LangC

Tipo	Descrição	Exemplo
num	Números inteiros	<code>num idade = 25;</code>
text	Cadeias de caracteres	<code>text nome = "Lucas";</code>
bool	Valores booleanos	<code>bool e = false;</code>

2.3 Estrutura do programa

Um programa LangC é composto por uma sequência de instruções. Toda instrução deve terminar com `;`. Não existem blocos, funções, laços, condicionais ou entrada de dados.

2.3.1 Declaração de variável

```
1 tipo identificador = valor_ou_expressao;
```

Listing 1: Formato de declaração

2.3.2 Saída com show

```
1 show expressao;
```

Listing 2: Formato de saída

3 Questão 1 – Tokens e expressões regulares

3.1 Definição dos tokens

A Tabela 2 apresenta todos os tokens definidos para a linguagem LangC, com suas respectivas expressões regulares.

Tabela 2: Tokens e expressões regulares da LangC

Token	Expressão regular	Descrição
NUM	(num)	Palavra reservada para inteiro
TEXT	(text)	Palavra reservada para texto
BOOL	(bool)	Palavra reservada para booleano
SHOW	(show)	Palavra reservada para saída
TRUE	(true)	Literal booleano verdadeiro
FALSE	(false)	Literal booleano falso
NUM_LIT	([0-9]+)	Literal numérico inteiro
CONST	("["^"]*")	Literal string entre aspas
VAR	([a-zA-Z_][a-zA-Z0-9_]*)	Identificador de variável
EQ_EQ	(==)	Operador de igualdade
EQ	(=)	Operador de atribuição
ADD	(\+)	Adição
SUB	(-)	Subtração
MUL	(*)	Multiplicação
DIV	(/)	Divisão
GT	(>)	Maior que
LT	(<)	Menor que
SEMICOLON	(;)	Delimitador de instrução
LPAREN	(\()	Parêntese esquerdo
RPAREN	(\))	Parêntese direito

3.2 Prioridade dos tokens

A ordem dos tokens no dicionário define a prioridade de reconhecimento. Palavras reservadas (NUM, TEXT, BOOL, SHOW, TRUE, FALSE) são declaradas **antes** de VAR, garantindo que a string `num` seja reconhecida como NUM e nunca como VAR. Da mesma forma, EQ_EQ vem antes de EQ para que `==` não seja reconhecido como dois tokens EQ.

3.3 Implementação

```

1 tokens = {
2     "NUM"      : r"(num)",
3     "TEXT"     : r"(text)",
4     "BOOL"     : r"(bool)",
5     "SHOW"     : r"(show)",
6     "TRUE"     : r"(true)",
7     "FALSE"    : r"(false)",
8     "NUM_LIT"  : r"([0-9]+)",
9     "CONST"    : r'"["^"]*"',
10    "VAR" (     : r"([a-zA-Z_][a-zA-Z0-9_]*)",
11    "EQ_EQ"     : r"(==)",
12    "EQ"        : r"(=)",

```

```

13  "ADD"      : r"(\+)",
14  "SUB"      : r"(-)",
15  "MUL"      : r"(\*)",
16  "DIV"      : r"(/)",
17  "GT"       : r"(>)",
18  "LT"       : r"(<)",
19  "SEMICOLON": r"(;)",
20  "LPAREN"   : r"(\()",
21  "RPAREN"   : r"(\))",
22  }

```

Listing 3: expressaoRegular.py – Tokens e expressões regulares

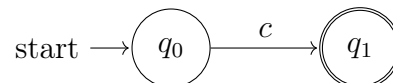
4 Questão 2 – Construção do NFA

4.1 Construção de Thompson

A Construção de Thompson é um algoritmo que converte uma expressão regular em um NFA. Para cada operação da expressão regular, existe uma construção correspondente de estados e transições.

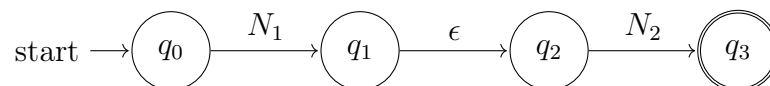
4.1.1 NFA básico para um caractere

Para um caractere c qualquer, o NFA é composto por dois estados e uma transição:

Figura 1: NFA para um caractere c

4.1.2 Concatenação

Dois NFAs N_1 e N_2 são concatenados ligando o estado final de N_1 ao estado inicial de N_2 por uma transição ϵ :

Figura 2: NFA para concatenação $N_1 \cdot N_2$

4.1.3 União

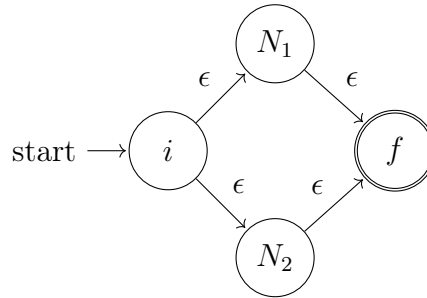


Figura 3: NFA para união $N_1 \mid N_2$

4.1.4 Fecho de Kleene (*) e Uma ou mais repetições (+)

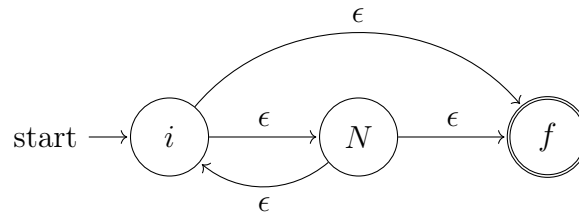


Figura 4: NFA para fecho de Kleene N^*

4.2 Unificação dos NFAs

Após construir um NFA individual para cada token, todos são unificados em um único NFA global através de um novo estado inicial com ϵ -transições para cada NFA individual:

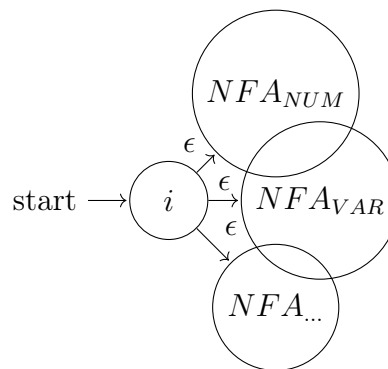


Figura 5: NFA global – estado inicial com ϵ -transições para cada token

4.3 Resultado

O NFA global construído para todos os 20 tokens da LangC possui **91 estados**. Abaixo estão os estados finais e seus tokens correspondentes:

Tabela 3: Estados finais do NFA global

Estado Final	Token	Prioridade
7	NUM	1
15	TEXT	2
23	BOOL	3
31	SHOW	4
39	TRUE	5
49	FALSE	6
53	NUM_LIT	7
61	CONST	8
67	VAR	9
71	EQ_EQ	10
73	EQ	11
75	ADD	12
77	SUB	13
79	MUL	14
81	DIV	15
83	GT	16
85	LT	17
87	SEMICOLON	18
89	LPAREN	19
91	RPAREN	20

5 Questão 3 – Conversão NFA para DFA

5.1 Algoritmo de subconjuntos

A conversão do NFA para DFA foi realizada pelo **Algoritmo de Subconjuntos** (*subset construction*). Cada estado do DFA corresponde a um **conjunto de estados** do NFA.

O algoritmo funciona da seguinte forma:

1. Calcula o ϵ -fechamento do estado inicial do NFA, tornando-se o estado inicial do DFA
2. Para cada estado do DFA (conjunto de estados do NFA) e cada símbolo do alfabeto:
 - Aplica a função *move*: encontra todos os estados NFA alcançáveis consumindo o símbolo
 - Calcula o ϵ -fechamento do resultado, torna-se um novo estado do DFA
3. Repete até não haver novos estados
4. Um estado do DFA é **final** se contiver pelo menos um estado final do NFA

5. Quando há conflito (mais de um token possível), o de **menor prioridade** é aplicado

5.2 Epsilon-Fechamento

```
1 def epsilon_closure(estados):
2     closure = set(estados)
3     stack = list(estados)
4     while stack:
5         estado = stack.pop()
6         if 'epsilon' in estado.transicoes:
7             for destino in estado.transicoes['epsilon']:
8                 if destino not in closure:
9                     closure.add(destino)
10                    stack.append(destino)
11 return frozenset(closure)
```

Listing 4: Função epsilon_closure

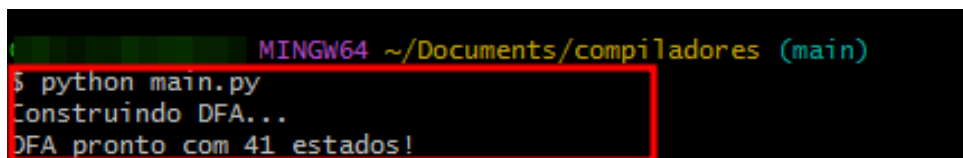
5.3 Resolução de conflitos por prioridade

Quando um estado do DFA contém múltiplos estados finais do NFA (ex: num pode ser NUM ou VAR), a prioridade definida na Questão 1 desempata:

```
1 for e_nfa in atual_dfa:
2     if e_nfa.is_final:
3         if e_nfa.prioridade < melhor_prioridade:
4             melhor_prioridade = e_nfa.prioridade
5             token_final = e_nfa.token
```

Listing 5: Resolução de conflito por prioridade

5.4 Resultado



```
MINGW64 ~/Documents/compiladores (main)
$ python main.py
Construindo DFA...
DFA pronto com 41 estados!
```

O DFA construído a partir do NFA global possui um número reduzido de estados em relação ao NFA, pois estados equivalentes são agrupados. Cada estado do DFA é representado como um `frozenset` de estados do NFA.

6 Questão 4 – Analisador léxico

6.1 Funcionamento

O analisador léxico utiliza o DFA para processar uma linha de código LangC e retornar a sequência de tokens reconhecidos. O algoritmo implementa a estratégia do **maior casamento possível** : avança o máximo possível no DFA antes de emitir um token.

1. Ignora espaços em branco
2. A partir da posição atual, caminha pelo DFA consumindo caracteres
3. Sempre que chega em um estado final, salva o token e a posição
4. Quando não há mais transições possíveis, emite o último token salvo
5. Reinicia do ponto salvo para reconhecer o próximo token
6. Se nenhum estado final foi alcançado, retorna ERRO

6.2 Validação adicional

Além do reconhecimento via DFA, o analisador implementa uma validação extra: identificadores (VAR) com mais de 30 caracteres são rejeitados com ERRO, conforme especificação da linguagem.

```
1 if token_encontrado == "VAR" and len(lexema) > 30:  
2     erro_na_linha = True  
3     break
```

Listing 6: Validação de tamanho do identificador

6.3 Interface com o usuário

O programa oferece duas formas de entrada:

- **Arquivo:** lê um arquivo `.txt` com o código LangC
- **Manual:** o usuário digita linha por linha e cada linha é processada imediatamente ao pressionar Enter

7 Testes e resultados

7.1 Casos de Teste

A Tabela 4 apresenta os casos de teste utilizados para validar o analisador léxico.

Tabela 4: Casos de teste e resultados esperados

Entrada	Saída Esperada
num a = 0 ;	NUM VAR EQ NUM_LIT SEMICOLON
num b = 5 + a ;	NUM VAR EQ NUM_LIT ADD VAR SEMICOLON
text c = "teSte";	TEXT VAR EQ CONST SEMICOLON
@teste = 5 ;	ERRO
boooooool d = true ;	VAR VAR EQ TRUE SEMICOLON
1valor = 3 ;	NUM_LIT VAR EQ NUM_LIT SEMICOLON
show a < b ;	SHOW VAR LT VAR SEMICOLON
show a == b ;	SHOW VAR EQ_EQ VAR SEMICOLON
num x = a \$ b ;	ERRO
show "Ola mundo";	SHOW CONST SEMICOLON
bool e = false ;	BOOL VAR EQ FALSE SEMICOLON
num% = 10 ;	ERRO
show (2 + 3) * 4 ;	SHOW LPAREN NUM_LIT ADD NUM_LIT RPAREN MUL NUM_LIT SEMICOLON
text nome = "Maria";	TEXT VAR EQ CONST SEMICOLON
num y = a + #b ;	ERRO
show true ;	SHOW TRUE SEMICOLON
show a > b ;	SHOW VAR GT VAR SEMICOLON
nome-completo = "ana";	VAR SUB VAR EQ CONST SEMICOLON
show? = 1 ;	ERRO

7.2 Análise dos resultados

7.2.1 Erros léxicos corretos

As entradas com caracteres inválidos do alfabeto da LangC (@, \$, #, %, ?) foram corretamente identificadas como ERRO.

7.2.2 Prioridade de palavras reservadas

A entrada `show a == b ;` foi corretamente reconhecida com `EQ_EQ` em vez de dois tokens `EQ`, demonstrando que a prioridade está funcionando. Da mesma forma, `bool e = false ;` produziu `BOOL` e não `VAR`.

7.2.3 Maior casamento possível

A entrada `numeral` foi corretamente reconhecida como `VAR` e não como `NUM` seguido de `VAR`, demonstrando que o algoritmo de maior casamento está funcionando corretamente.

7.2.4 Erros semânticos não tratados no léxico

Casos como `text msg = 42 ;` (tipo incompatível) e `num bool = 5 ;` (palavra reservada como variável) produzem sequências de tokens válidas. Esses erros são de natureza **semântica** e estão fora do escopo do analisador léxico.

```
Escolha (1, 2 ou 3): 2

Modo manual. Digite 'sair' para voltar ao menu.

> text msg = 42 ;
TEXT VAR EQ NUM_LIT SEMICOLON
> num bool = 5 ;
NUM BOOL EQ NUM_LIT SEMICOLON
```

7.3 Saída completa dos testes

```
--- Resultado ---
NUM VAR EQ NUM_LIT SEMICOLON
NUM BOOL EQ NUM_LIT ADD VAR SEMICOLON
TEXT VAR EQ CONST SEMICOLON
ERRO
VAR VAR EQ TRUE SEMICOLON
NUM_LIT VAR EQ NUM_LIT SEMICOLON
SHOW VAR LT VAR SEMICOLON
SHOW VAR EQ_EQ VAR SEMICOLON
ERRO
SHOW CONST SEMICOLON
BOOL VAR EQ FALSE SEMICOLON
ERRO
SHOW LPAREN NUM_LIT ADD NUM_LIT RPAREN MUL NUM_LIT SEMICOLON
TEXT VAR EQ CONST SEMICOLON
ERRO
SHOW TRUE SEMICOLON
NUM VAR EQ VAR ADD VAR SEMICOLON
SHOW VAR GT VAR SEMICOLON
TEXT VAR EQ NUM_LIT SEMICOLON
NUM VAR EQ LPAREN NUM_LIT DIV NUM_LIT RPAREN SEMICOLON
SHOW FALSE SEMICOLON
VAR SUB VAR EQ CONST SEMICOLON
NUM VAR EQ VAR MUL VAR SEMICOLON
SHOW VAR SEMICOLON
NUM VAR EQ VAR SUB VAR SEMICOLON
ERRO
SHOW VAR SEMICOLON
-----
```

```

1 NUM VAR EQ NUM_LIT SEMICOLON
2 NUM BOOL EQ NUM_LIT ADD VAR SEMICOLON
3 TEXT VAR EQ CONST SEMICOLON
4 ERRO
5 VAR VAR EQ TRUE SEMICOLON
6 NUM_LIT VAR EQ NUM_LIT SEMICOLON
7 SHOW VAR LT VAR SEMICOLON
8 SHOW VAR EQ_EQ VAR SEMICOLON
9 ERRO
10 SHOW CONST SEMICOLON
11 BOOL VAR EQ FALSE SEMICOLON
12 ERRO
13 SHOW LPAREN NUM_LIT ADD NUM_LIT RPAREN MUL NUM_LIT SEMICOLON
14 TEXT VAR EQ CONST SEMICOLON
15 ERRO
16 SHOW TRUE SEMICOLON
17 NUM VAR EQ VAR ADD VAR SEMICOLON
18 SHOW VAR GT VAR SEMICOLON
19 TEXT VAR EQ NUM_LIT SEMICOLON
20 NUM VAR EQ LPAREN NUM_LIT DIV NUM_LIT RPAREN SEMICOLON
21 SHOW FALSE SEMICOLON
22 VAR SUB VAR EQ CONST SEMICOLON
23 NUM VAR EQ VAR MUL VAR SEMICOLON
24 SHOW VAR SEMICOLON
25 NUM VAR EQ VAR SUB VAR SEMICOLON
26 ERRO
27 SHOW VAR SEMICOLON

```

Listing 7: Saída do analisador léxico para os casos de teste

8 Estrutura do projeto

Tabela 5: Arquivos do projeto

Arquivo	Responsabilidade
<code>expressaoRegular.py</code>	Q1 – Tokens e expressões regulares
<code>geradorNFA.py</code>	Q2 – Construção de Thompson + parser de regex
<code>geradorDFA.py</code>	Q3 – Algoritmo de subconjuntos (NFA → DFA)
<code>analisadorLexico.py</code>	Q4 – Analisador léxico completo
<code>main.py</code>	Ponto de entrada com interface ao usuário
<code>tokens.txt</code>	Lista de tokens e descrições
<code>testes_lexico.txt</code>	Casos de teste

9 Como executar

9.1 Requisitos

- Python 3.8 ou superior
- Nenhuma biblioteca externa necessária

9.2 Arquivos necessários

Todos os arquivos devem estar na **mesma pasta**:

- `main.py`
- `analizadorLexico.py`
- `geradorDFA.py`
- `geradorNFA.py`
- `expressaoRegular.py`
- `testes_lexico.txt` (necessário para a opção 1)

9.3 Execução

Para executar o programa, basta rodar o arquivo `main.py`:

```
1 python main.py
```

Listing 8: Comando de execução

Ao iniciar, o programa constrói automaticamente o DFA a partir das expressões regulares e exibe o menu principal:

```
1 Construindo DFA...
2 DFA pronto!
3
4 Como deseja fornecer o codigo?
5 1 - Carregar arquivo
6 2 - Digitar manualmente
7 3 - Sair
8
9 Escolha (1, 2 ou 3):
```

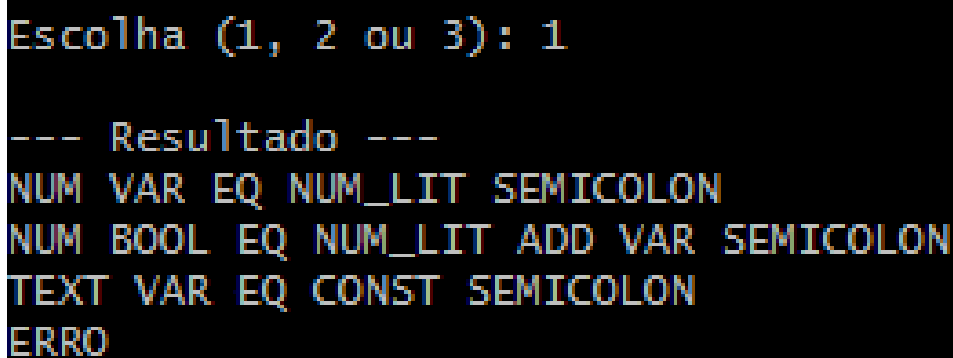
Listing 9: Menu principal

9.4 Opção 1 – Carregar arquivo

Ao escolher a opção **1**, o programa lê automaticamente o arquivo `testes_lexico.txt` e exibe o resultado de cada linha:

```
1 Escolha (1, 2 ou 3): 1
2
3 --- Resultado ---
4 NUM VAR EQ NUM_LIT SEMICOLON
5 NUM VAR EQ NUM_LIT ADD VAR SEMICOLON
6 TEXT VAR EQ CONST SEMICOLON
7 ERRO
8 ...
9 -----
```

Listing 10: Execução com arquivo



```
Escolha (1, 2 ou 3): 1
--- Resultado ---
NUM VAR EQ NUM_LIT SEMICOLON
NUM BOOL EQ NUM_LIT ADD VAR SEMICOLON
TEXT VAR EQ CONST SEMICOLON
ERRO
```

9.5 Opção 2 – Modo manual

Ao escolher a opção **2**, o programa entra no modo interativo. Cada linha digitada é processada imediatamente ao pressionar **Enter**. Para voltar ao menu, digite **sair**:

```
1 Escolha (1, 2 ou 3): 2
2
3 Modo manual. Digite 'sair' para voltar ao menu.
4
5 > num a = 5 ;
6 NUM VAR EQ NUM_LIT SEMICOLON
7 > show a + b ;
8 SHOW VAR ADD VAR SEMICOLON
9 > @erro
10 ERRO
11 > show a == b ;
12 SHOW VAR EQ_EQ VAR SEMICOLON
13 > sair
14
15 Como deseja fornecer o codigo?
```



```
16 1 - Carregar arquivo
17 2 - Digitar manualmente
18 3 - Sair
```

Listing 11: Execução no modo manual

```
Escolha (1, 2 ou 3): 2

Modo manual. Digite 'sair' para voltar ao menu.

> num a = 5 ;
NUM VAR EQ NUM_LIT SEMICOLON
> show a + b ;
SHOW VAR ADD VAR SEMICOLON
> @erro
ERRO
> show a == b ;
SHOW VAR EQ_EQ VAR SEMICOLON
> sair
Como deseja fornecer o código?
1 - Carregar arquivo
2 - Digitar manualmente
3 - Sair
```

9.6 Opção 3 – Sair

Encerra o programa.

```
1 Escolha (1, 2 ou 3): 3
2 Encerrando...
```

Listing 12: Encerrando o programa

10 Conclusão

O analisador léxico para a linguagem LangC foi implementado com sucesso seguindo o pipeline completo: expressões regulares $\rightarrow$ NFA (Thompson) $\rightarrow$ DFA (subconjuntos) $\rightarrow$ analisador léxico.

Os principais desafios foram:

- Implementar o parser de expressões regulares.
- Garantir a prioridade correta entre tokens para que palavras reservadas nunca sejam reconhecidas como identificadores

- Implementar o algoritmo de maior casamento (*maximal munch*) no analisador léxico para reconhecer corretamente tokens como == em vez de dois =

Todos os casos de teste produziram os resultados esperados, com erros léxicos corretamente identificados e tokens válidos reconhecidos com a prioridade adequada.